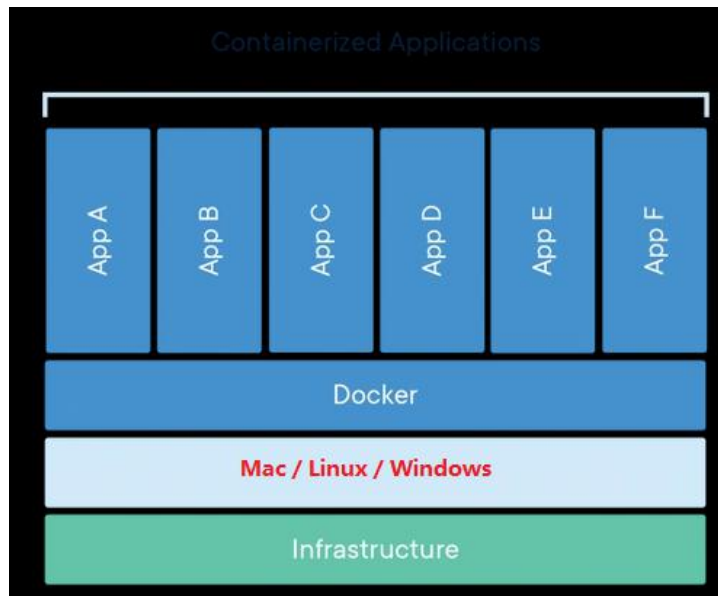


Docker 学习笔记

（在这之前，请先查看 Linux 知识的相关笔记）

一、基本概念

docker 是一种容器化技术，通过安装 docker 应用，可以在任何平台提供 docker 环境，运行内部应用。



1. docker 的基本要素：

（1）Docker Engine

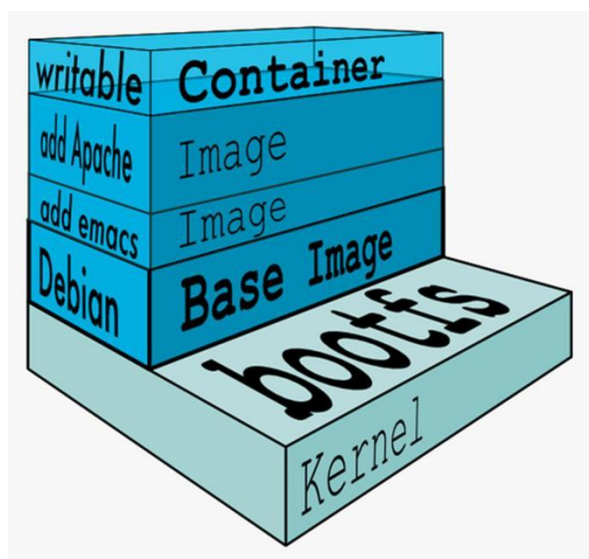
利用宿主机的 Linux 内核，提供镜像构建、容器运行等 Docker 功能，如果宿主机不是 Linux 系统，则会安装一个 Linux 内核供 Docker 使用。

（2）镜像 （Image）

采用分层文件系统，提供了一个简单的 Linux 系统的用户空间（如：centos 镜像、Debian 镜像），并在此基础上，添加了用户所需的软件、环境、启动参数等，是运行容器的模板文件。

（3）容器 （Container）

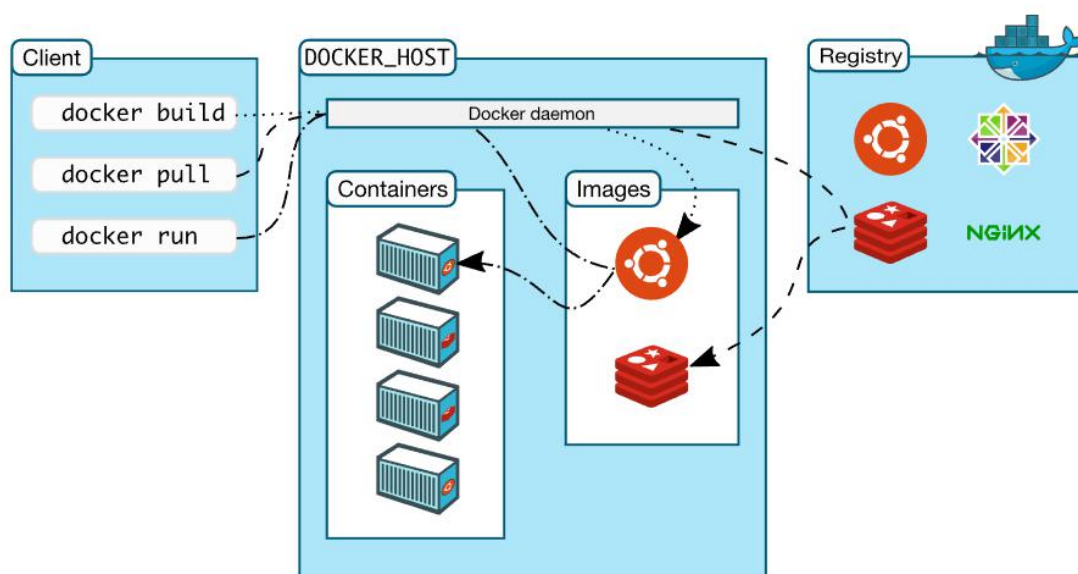
容器是镜像的运行实例，利用 Linux 内核，结合镜像提供的 Linux 用户空间，将镜像运行起来，每一个容器都可以理解为一个独立简单的 Linux 系统。



(4) 仓库 (Registry)

负责镜像管理的中央仓库，用户可以拉取自己所需的镜像，也可以推送已经打包好的镜像；

默认使用官方 DockerHub 的，也可以设置成其他的或者搭建自己的仓库。



2. 和虚拟机的区别

每一个虚拟机在创建时，需要指定消耗宿主机的内存、CPU 等资源的值，且每一个虚拟机都必须有一个完整的操作系统（包括内核），资源消耗大；

Docker 各个容器则共享一个系统内核，共享操作系统的资源，各个容器写隔离且只有一个简单的操作系统用户空间，只包含运行所需要的必要文件，资源消耗大大降低；



二、docker 详解

1. 依赖的 Linux 技术

Docker 镜像的构建以及容器的运行都需要宿主机的 Linux 内核的技术支持，依赖的技术有：隔离技术（namespace）、资源控制技术（Cgroups）、分层文件系统（UnionFS）

（了解下面的，先看 Linux 知识的网络部分）

（1）隔离技术（namespace）

docker 在运行容器时，会利用 linux 提供的 namespace 技术，为每一个容器创建独立的空间，拥有独立的网络、进程、文件操作系统等，每一个容器相当于拥有了一个独立的 Linux 用户空间。

比如两个容器可以有相同的文件目录、进程 ID 等

以网络空间结合 Docker 网络模式举例：

① Docker 的常用网络模式

```
root@localhost:~# docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
dbf6b4085225        bridge             bridge              local
86b3eeb8c1c7        host               host                local
28a0f00b15c9        none               null                local
e5664b1e0ea0        tomcat-net         bridge              local
```

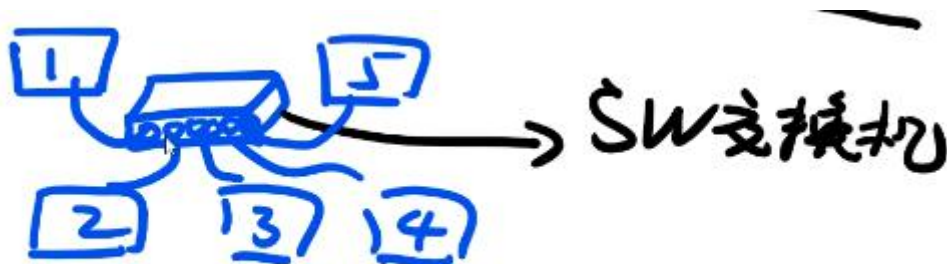
1) Bridge 模式

Docker 容器的默认模式,在运行容器时，如果不指定网络模式，会默认使用 bridge 模式。

在安装 docker 程序时，会在宿主系统上默认的网络空间（root network namespace）创建一张虚拟网卡，有默认的网段

在 Bridge 网络模式下，容器的 IP 基于该网段生成，Linux 系统中会以 docker 前缀进行命名，比如 docker0；

其本质是一个网桥（bridge），相当于一个交换机

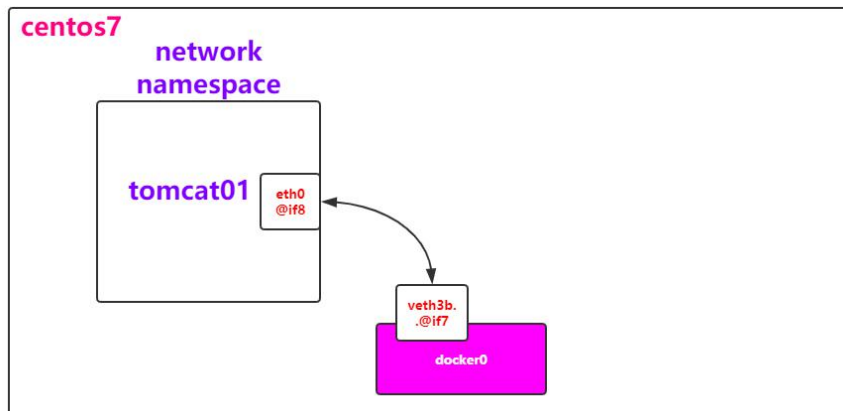


```

3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue sta
   link/ether 52:54:00:b1:82:52 brd ff:ff:ff:ff:ff:ff
   inet 192.168.122.1/24 brd 192.168.122.255 scope global virbr0
      valid_lft forever preferred_lft forever
4: virbr0-nic: <BROADCAST,MULTICAST> mtu 1500 qdisc pfifo_fast master vir
   link/ether 52:54:00:b1:82:52 brd ff:ff:ff:ff:ff:ff
5: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue st
   link/ether 02:42:3a:f9:de:9d brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0

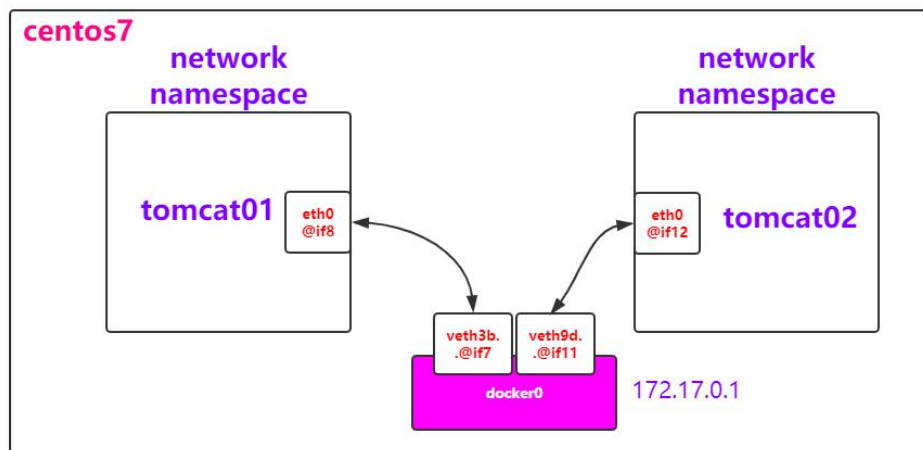
```

docker 每运行一个容器，docker 就会给该容器创建一个新的网络空间（network namespace），使用 veth pair 将它和默认网络空间连接，同时分别在新的和默认的网络空间创建虚拟网卡，并做初始 IP 等设置，保证能正常通信；

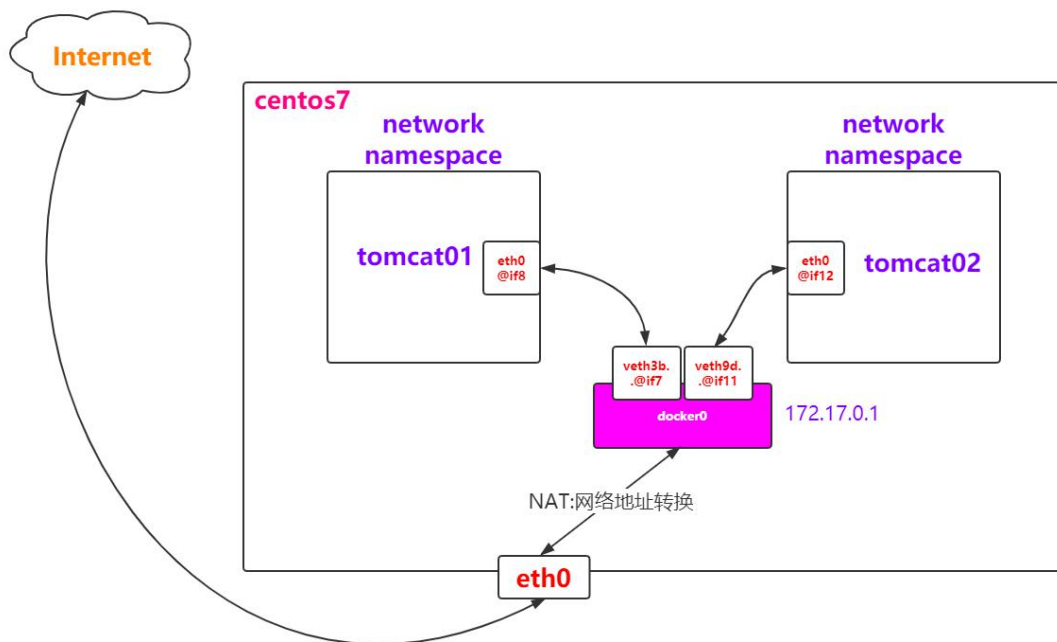


容器默认使用初始网桥，保证各个容器之间可以正常通信。

容器之间相互通信时，会先连接宿主主机上的 docker 网桥，由该网桥转发到目标容器的网络空间中。



容器同宿主机外进行通信，会先接入网桥，然后用 NAT 转发，使用宿主机的 IP 和端口，让容器和宿主机外进行通信；



运行容器时，如果不指定端口映射

容器访问宿主机外时，使用内核随机分配一个端口进行访问；

宿主机外访问容器时，则无法访问；

所以，如果宿主机外要访问容器，在运行时需要指定端口映射，使用宿主机的 IP 和映射的端口即可访问容器；

```
root@localhost:~# docker run -d -p 8080:80 --name my-nginx nginx
```

我们也可以创建自定义网桥，在容器运行时指定使用该网桥

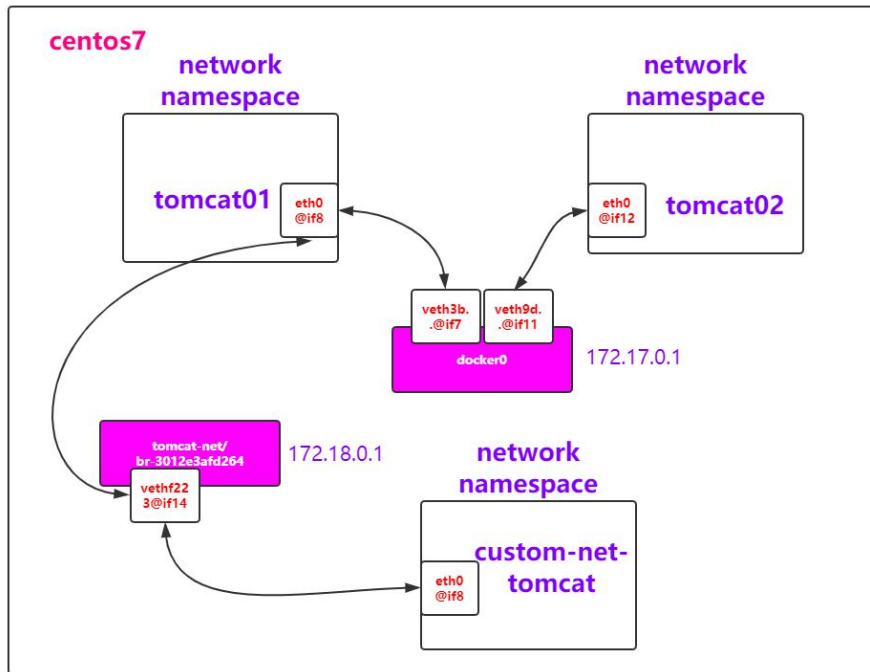
每新建一个网桥，默认会分配一个新的网段

```
root@localhost:~# docker network create tomcat-net
e5664b1e0ea01461403b193ab3497a8be91f83f0c1becd43ddb389c3b7b7815f
root@localhost:~# docker run -d --network tomcat-net tomcat
```

并且新建的网桥也可以通过 NAT 转发，使用宿主机 IP

```
root@localhost:~# docker exec -it 1f8c7edf48db bash
root@1f8c7edf48db:/# curl www.baidu.com
<!DOCTYPE html>
<!--STATUS OK--><html> <head><meta http-equiv=content-type conte
http://s1.bdstatic.com/r/www/cache/bdorz/baidu.min.css><title>百
r> <div id=lg> <img hidefocus=true src=//www.baidu.com/img/bd lo
```

连接不同的网桥的容器，处于不同的网段中，无法进行通信。可以通过 docker 命令，建立网桥和另一个网桥容器的连接，来完成所属不同网桥容器之间的通信



```
root@1f8c7ed148db:/# exit
exit
root@localhost:~# docker network connect tomcat-net tomcat01
```

a. 自定义网桥和 Link 命令

在 docker 中，默认网桥不提供容器的 DNS 服务，但是自定义网桥提供容器的 DNS 服务，即容器名和 IP 的映射服务

在自定义网桥中，可以直接使用容器名代替 IP，就能完成对容器的访问

```
root@localhost:~# docker exec -it priceless_hawking ping bold_ritchie
```

如果一定要使用默认网桥，可以手动使用 Link 命令建立 DNS

在生产环境中，推荐使用自定义网桥来建立容器服务，这样我们就可以在 java 等服务的配置文件中的 IP 部分直接填写容器名即可

```
docker run -d --name mysql01 --network mysql-network mysql
```

```
<> YAML
url: jdbc:mysql://mysql_container:3306/testdb
```

2) None 模式

需要在运行容器时指定

容器在 none 模式下，有自己的网络空间（network namespace），但没有网络功能，不具有和容器外进行网络通信的能力

```

[root@localhost ens33]# ip netns exec ns1 ip link set lo up
[root@localhost ens33]# ip netns add ns2
[root@localhost ens33]# docker run --network none -it busybox

```

3) Host 模式

需要在运行容器时指定，性能最高，但是慎用

```

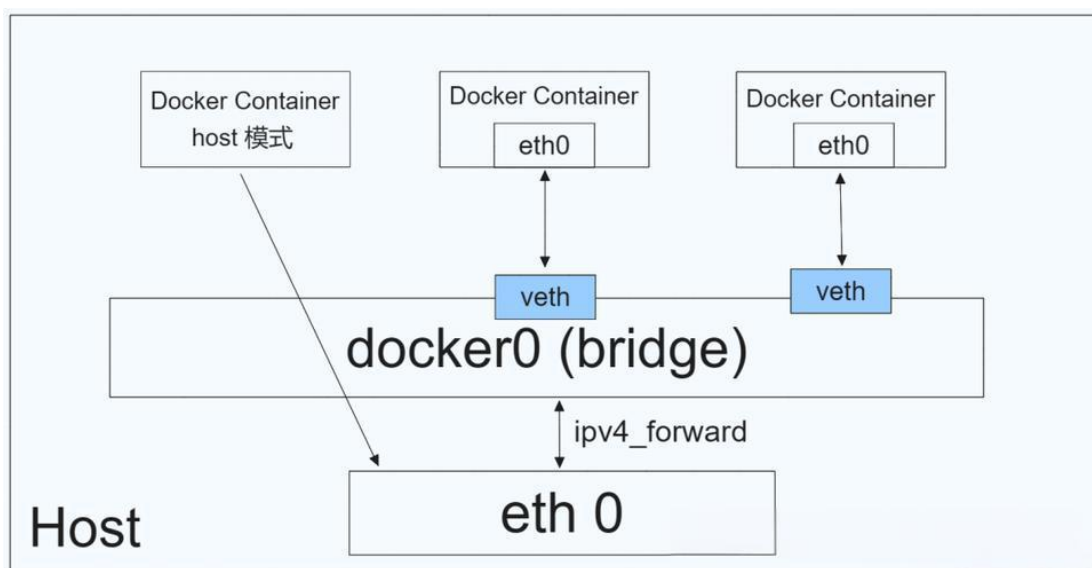
docker run -d --network host nginx:latest

```

容器不再和宿主机做网络隔离，没有自己的网络空间，直接使用宿主机的网络空间，但是文件系统，进程等依旧使用 namespace 技术进行隔离；

IP 和宿主机一样，端口也直接使用宿主机的，不需要做端口映射；

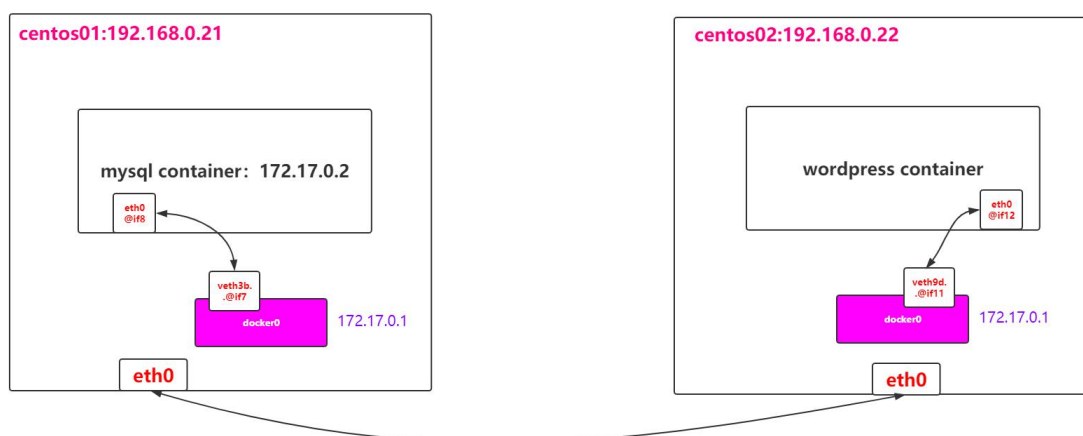
适合宿主机只部署一个容器的情况，如 MySQL；



4) Overlay 模式

Overlay 模式是 Docker Swarm 和 K8S 等容器集群服务程序默认用的网络模式，实现多台宿主机上的容器之间通信。

需要注意的是，通信双方的容器可以不处于同一个网段，但是 IP 一定不能冲突



(2) Cgroups (资源隔离)

Cgroups 是 Linux 内核提供的对硬件访问的限制技术，Cgroups 同样做了很多细分，如 CPU 核心数、网络的带宽、内存使用量等，不同的资源限制技术 Linux 有不同时实现机制。

Cgroups 是基于进程来工作的，一个容器对应宿主机的一个或多个进程，宿主机根据进程号（PID）进行资源限制

cgroup 底层技术原理一览表

子系统	作用	限制方式	主要控制的资源
 cpu	限制 CPU 使用率和调度控制	通过控制进程在 CFS 中的调度时间片比例和配额实现限制	CPU 使用率 CPU 时间片
 memory	限制内存使用量	设置内存使用上限，超出后触发内存回收或 OOM (Out Of Memory) 杀死进程	物理内存 (可包含 Swap)
 blkio	限制块设备 I/O 访问	通过 I/O 权重和限流控制读写速度和 I/O 操作数	磁盘 I/O (读写速度、IOPS)
 net_cls	标记网络数据包 (配合流量控制)	为进程打上 classid 标签，配合 tc (traffic control) 实现流量控制	网络流量分类 (不直接限速)
 net_prio	设置网络优先级	为网络流量设置优先级，配合 tc 实现优先级调度	网络流量优先级
 pids	限制进程数量	设置最大进程数，超出则无法创建新进程	进程数量

容器在启动时，默认是使用宿主机的全部硬件资源，可以使用参数指定容器使用的硬件资源的上限，如：最大使用内存

```
root@localhost:~# docker run -d --name mycontainer --memory="512m"
```

最大内存

(3) UnionFS （分层文件系统）

(这里需要先了解 Linux 的文件系统知识，见笔记)

docker 默认使用的是 Linux 内核提供的 OverlayFS 分层系统，也可以其他 UnionFS 技术，需要在配置文件 daemon 中指定：


```
root@localhost:~# cat /etc/docker/daemon.json
{
  "registry-mirrors": ["https://ugmqx.dqzboy.xyz"]
}
root@localhost:~#
```

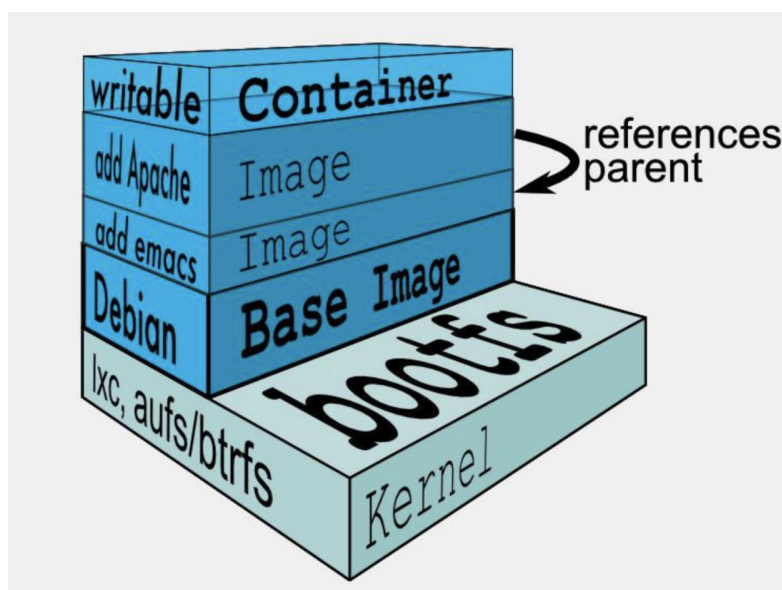
① Docker 层概念

Docker 使用的是宿主机的内核，在此基础上进行相应的 docker 操作。

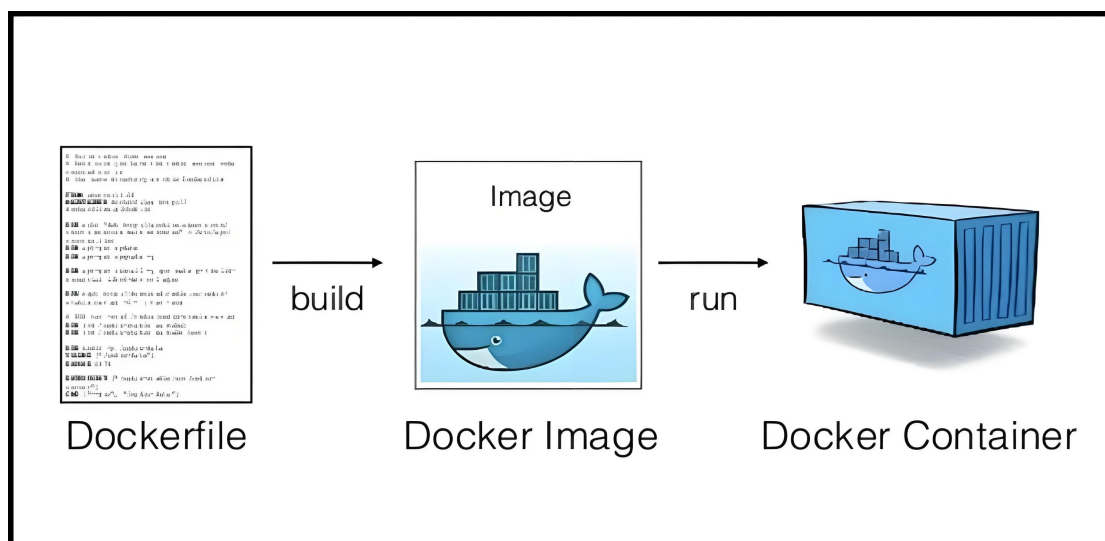
在 Linux 系统一切皆文件，即 docker 使用的是宿主机的 `bootfs` 文件系统来使用内核功能；在日常开发使用中，镜像的最底层，是 Linux 不同版本（centos、ubuntu 等）的用户文件系统 `rootfs`，通常只保留了 docker 必需的文件大概只有几十兆左右；

宿主机的 `bootfs` 和镜像部分的 `rootfs` 组成了最小可运行的 Linux 操作系统。

在 `rootfs` 基础上，镜像还会添加软件、一些 Linux 指令（比如环境变量设置），以层的方式，按照先后顺序打包在一起，以保证在 `rootfs` 运行后，达到用户所需环境；

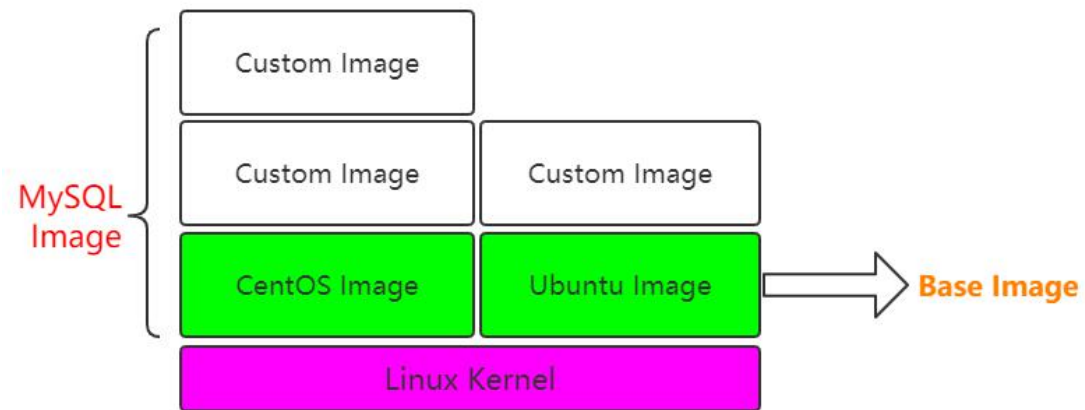


Dockerfile 文件是镜像构建的常用方法之一，可以用编译的方式，将基础镜像、Linux 指令等以层级的方式来构建用户镜像；



在 docker 中，默认使用 OverlayFS 对文件系统进行堆叠，只有最上层是读写层，下面的

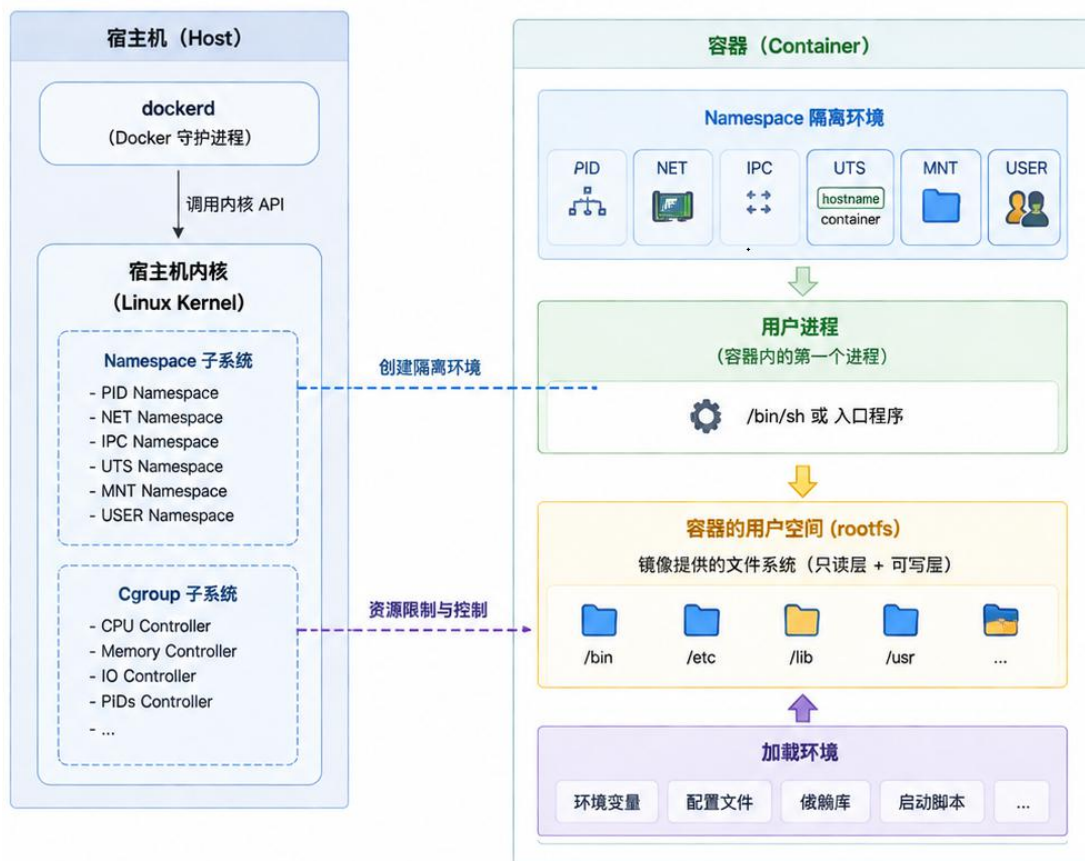
层都是只读层；



在运行容器时，宿主机内核启动一个进程，并利用 namespace 创建独立的运行环境，使用 cgroup 对该进程进行资源控制；

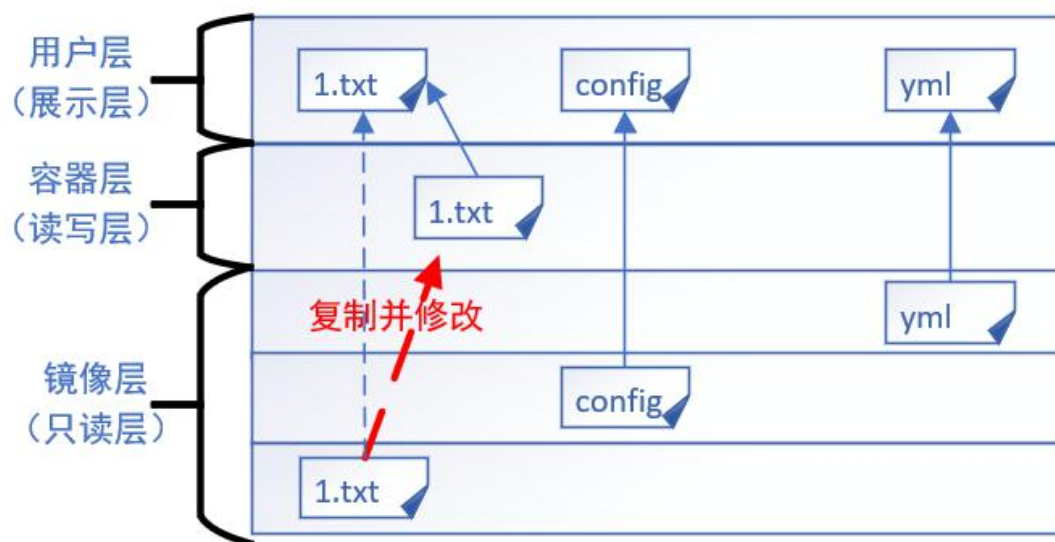
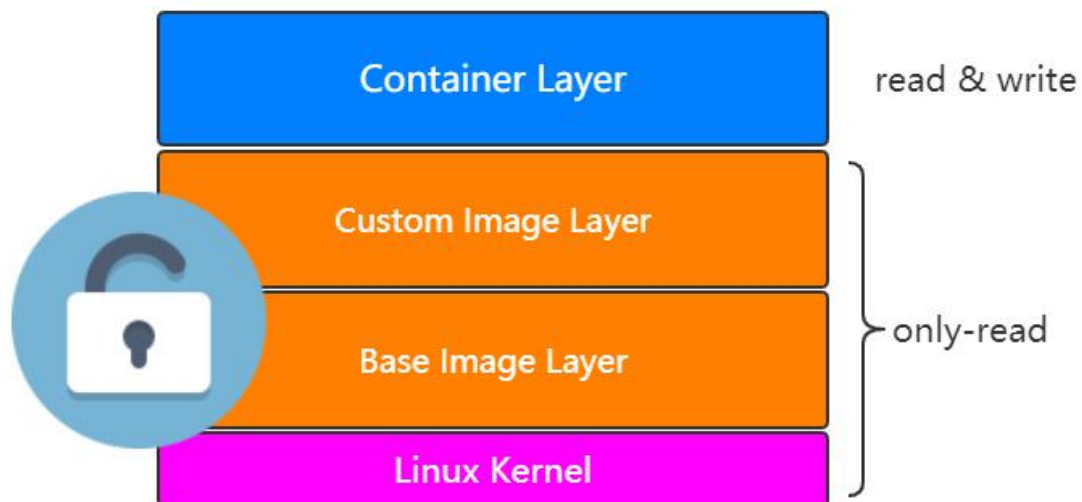
然后会在 namespace 里，利用宿主机内核，创建用户进程并运行镜像提供的用户空间 rootfs，并加载软件、环境变量语句等；

这样一个容器进程就运行起来了；



文件系统方面，会利用 UnionFS 技术，将镜像层的文件设置为只读层，并且上面创建一个读写层，即我们所说的容器层，负责文件的修改操作以及修改后文件的存储；

用户在进入容器后，其实看到的是 UnionFS 的展示层（即合并后的文件系统视图），当用户要对目录文件的增删改等修改操作都需要使用写时复制（copy-on-write），在容器层进行并存储。



容器也可以打包成镜像，如果在容器中进行了修改，如下载了新的软件，配置了新的环境，可以利用 `docker` 的 `commit` 命令，将容器层变为镜像层，打包成镜像，便于移植。

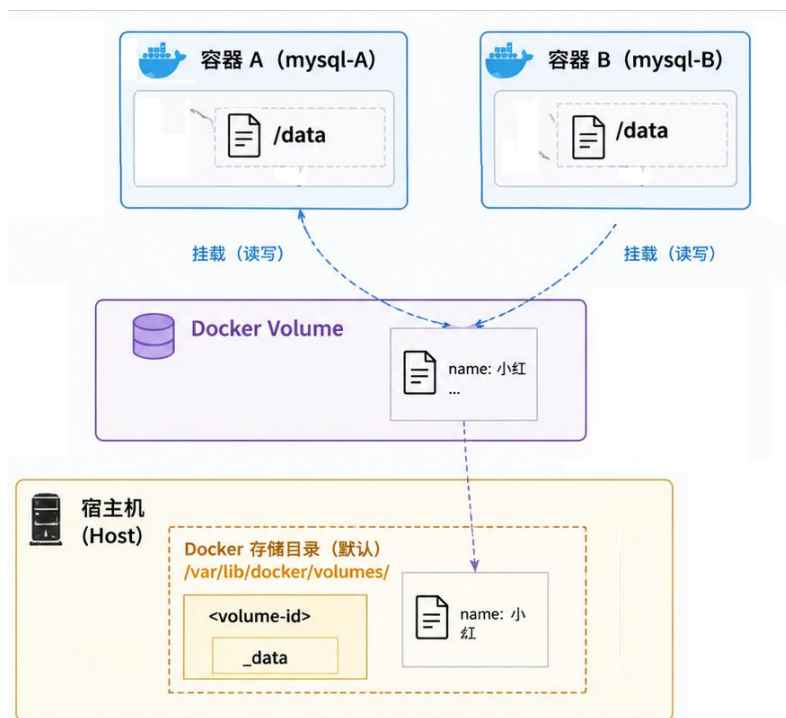
```
docker commit [OPTIONS] CONTAINER_ID IMAGE_NAME[:TAG]
```

2. Docker 应用

(1) 数据持久化技术-卷 (VOLUME)

`Docker` 的卷可以将容器指定目录的数据持久化到磁盘目录中，当容器被删除时，数据依然保存在磁盘上；

卷支持容器共享，多个容器可以使用同一个卷，但是要注意数据冲突；



卷可以在运行容器时使用参数指定，如果误删容器，可以重新启动将卷在磁盘上的数据加载到容器中

```
root@localhost:~# docker volume create my_volume
my_volume
root@localhost:~# docker volume ls
DRIVER      VOLUME NAME
local       my_volume
root@localhost:~# docker run -d -v my_volume:/data my_image
Unable to find image 'my_image:latest' locally
```

容器目录

具体使用步骤（了解）：

- ① 首先创建卷在磁盘的映射目录（有默认值）
 - ② 然后在容器运行时指定卷要持久化容器的哪个目录
- （2）具名挂载（Bind Mounting）

Bind Mounting 也是一种容器持久化技术，直接在运行容器时，填入磁盘路径和要同步时容器路径。

```
root@localhost:~# docker run -d -v /tmp/test:/usr/local/tomcat/webapps/test tomcat
```

磁盘目录 容器目录

这样，容器中对路径下的数据就会持久化到磁盘中；



(3) Docker compose

在实际的生产实践中，比如我们在启动一个容器时，参数中要填写端口映射、卷的挂载、以及资源限制等信息，为了协调各个容器的通信，还必须保证它们在同一个网桥中。容器一多，启动和管理便成了比较麻烦的问题；

为了方便容器、网桥、卷等的批量创建和使用，docker 推出了 compose 功能，可以在 yml 文件中，将版本信息、容器、网桥、卷等创建所需的参数统一配置，然后使用 docker compose up 命令，运行该 yml 文件统一创建，默认使用 docker-compose.yml 运行

1 编写 docker-compose.yml

```
docker-compose.yml

version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    volumes:
      - ./html:/usr/share/nginx/html
    networks:
      - app-net
    deploy:
      resources:
        limits:
          cpus: '0.50'
          memory: 256M
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: example
    volumes:
      - db_data:/var/lib/mysql
    networks:
      - app-net
    deploy:
```

2 执行 Docker Compose 命令

```
> docker-compose up -d
```

执行命令

Docker Compose

- 解析 docker-compose.yml
- 检查并拉取镜像
- 创建网络、卷等资源
- 创建并启动容器

调用 Docker API

3 Docker Engine 创建资源

网络 (Network)

app-net (bridge)
172.18.0.0/16

卷 (Volume)

db_data
(持久化存储)

镜像 (Images)

nginx:latest
mysql:8.0


```

version: '3.8' → 版本信息
services: → 容器
# Web 服务容器
web:
  image: nginx:latest
  container_name: web_container
  ports:
    - "8080:80" # 将宿主机 8080 端口映射到容器内的 80 端口
  volumes:
    - web_data:/usr/share/nginx/html # 将宿主机的 web_data 卷挂载到容器的目录
  networks:
    - my_bridge_network # 让容器加入同一个网桥网络
  deploy:
    resources:
      limits:
        cpus: "0.5" # 限制容器使用最多 0.5 个 CPU 核心
        memory: "512M" # 限制容器最多使用 512MB 内存

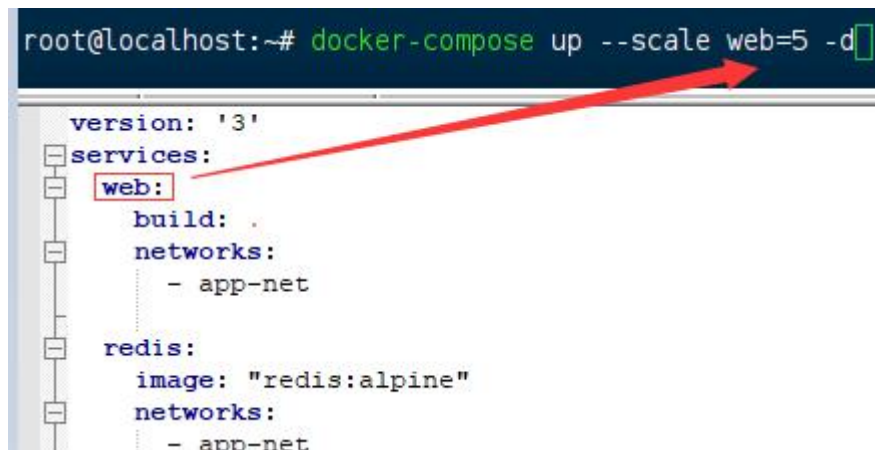
# 数据库容器
db:
  image: mysql:5.7
  container_name: db_container
  environment:
    MYSQL_ROOT_PASSWORD: example
  ports:
    - "3306:3306" # 将宿主机 3306 端口映射到容器的 3306 端口
  volumes:
    - db_data:/var/lib/mysql # 将宿主机的 db_data 卷挂载到容器的目录
  networks:
    - my_bridge_network # 让容器加入同一个网桥网络
  deploy:
    resources:
      limits:
        cpus: "1.0" # 限制容器使用最多 1 个 CPU 核心
        memory: "1G" # 限制容器最多使用 1GB 内存

# 定义一个网络：自定义网桥网络
networks: → 网桥
  my_bridge_network:
    driver: bridge # 使用 bridge 网络模式

# 定义卷：持久化容器数据
volumes: → 卷
  web_data:
    driver: local # 使用默认的本地卷驱动
  db_data:
    driver: local # 使用默认的本地卷驱动

```

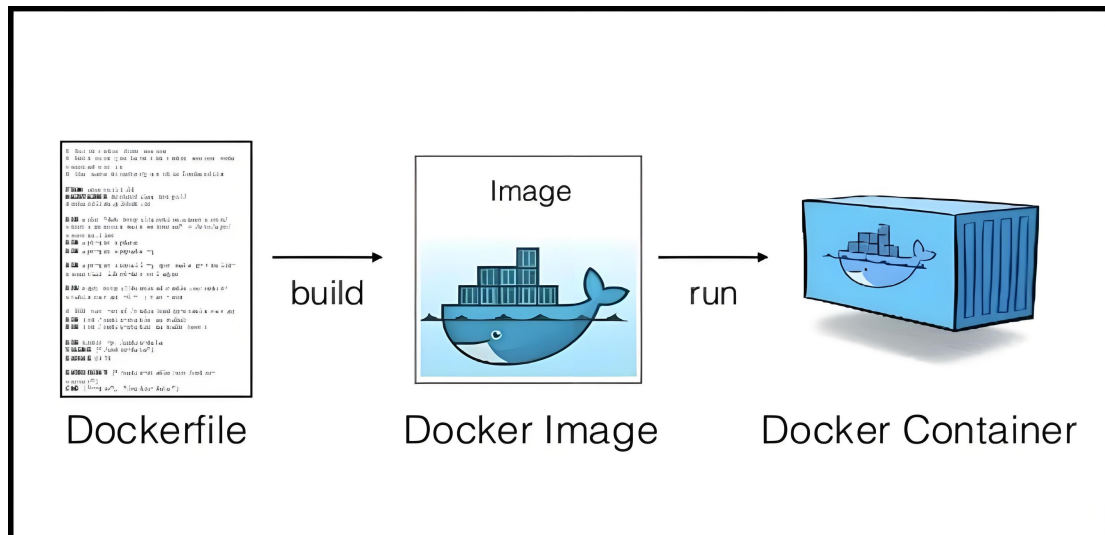
compose 的 yml 文件中，service 项用于配置要创建的容器需要的参数，默认情况下每有一个 service 子项就会创建一个容器实例，如果创建多个实例，则需要执行 docker compose up 命令时，使用参数 scale 来指定数量；



(4) DockerFile

构建镜像的方式:

- ① 运行已有的镜像为容器，修改容器内容并打包成镜像
- ② 编辑 dockerFile 文件，Dockerfile 定义了镜像的构建步骤、基础镜像、环境变量、依赖包等，通过 docker build 命令，按照顺序构建 dockerFile 文件生成镜像



3. Docker Swarm

很少用了，详见 K8S

4. Docker 常用命令

(1) 镜像类

- ① 镜像拉取 : docker pull
- ② 镜像推送: docker push
- ③ 镜像构建: docker build
- ④ 镜像运行容器: docker run
- ⑤ 镜像列表: docker images
- ⑥ 镜像删除: docker rmi

(2) 容器类

- ① 容器浏览: docker ps
- ② 容器删除: docker rm
- ③ 进入容器: docker exec -it

④ 容器构建为镜像: `docker commit`

(3) Compose

① 运行 compose 文件: `docker compose up`